Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

10. Dynamic Programming: 0/1 Knapsack Problem

Problem

Implement the 0/1 Knapsack problem using dynamic programming. Given the weight and value of each item and a knapsack capacity, determine the maximum total value without exceeding the capacity. Each item can be selected at most once.

Method

Use a bottom-up dynamic programming table **dp[i][w]**, representing the maximum value obtainable using the first **i** items with capacity **w**. For each item, either exclude it or include it if its weight fits. The recurrence is:

```
dp[i][w] = dp[i-1][w]                                          if weight[i-1] > w
 dp[i][w] = max(dp[i-1][w], value[i-1] + dp[i-1][w-weight[i-1]]) otherwise
```

Output:

Sample input: 4 items, capacity 7; weights = 1 3 4 5; values = 1 4 5 7.

```
$ ./knapsack
Enter Number of Items and Knapsack Capacity: Enter Weights of Items:
Enter Values of Items:
Maximum Value: 9
Selected Items (1-based): 2 3
```

Code:

```
/*
Program Name: 0/1 Knapsack Using Dynamic Programming
Program Description: Finds the maximum value that fits within a knapsack capacity.
Program Creation Time: Oct 30th, 2026
Program Creator: Ashwath Raj | AP25110010712
*/
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main(void) {
    int n, capacity;
    cout << "Enter Number of Items and Knapsack Capacity: ";
    cin >> n >> capacity;

    vector<int> weight(n), value(n);
    cout << "Enter Weights of Items:" << endl;
    for (int i = 0; i < n; i++) cin >> weight[i];

    cout << "Enter Values of Items:" << endl;
    for (int i = 0; i < n; i++) cin >> value[i];

    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i - 1][w];
            if (weight[i - 1] <= w) {
                dp[i][w] = max(dp[i][w],
                               value[i - 1] + dp[i - 1][w - weight[i - 1]]);
            }
        }
    }

    cout << "Maximum Value: " << dp[n][capacity] << endl;
    cout << "Selected Items (1-based):";
    int w = capacity;
    vector<int> selected;
    for (int i = n; i > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            selected.push_back(i);
```

```
            w -= weight[i - 1];
        }
    }
    reverse(selected.begin(), selected.end());
    if (selected.empty()) cout << " None";
    else for (int item : selected) cout << " " << item;
    cout << endl;
    return 0;
}
```

Complexity

Time: O(nW), where n is the number of items and W is the knapsack capacity.
Space: O(nW) for the dynamic programming table.